

|                                                                                   |                            |  |              |
|-----------------------------------------------------------------------------------|----------------------------|--|--------------|
|  | <b>Atividade — Aula 32</b> |  |              |
|                                                                                   | <b>NOME:</b>               |  |              |
|                                                                                   | <b>TURMA:</b>              |  |              |
|                                                                                   | <b>TURNO:</b>              |  | <b>NOTA:</b> |
|                                                                                   | <b>DATA:</b>               |  |              |

## Parte 1 — Conceitos

**1.** Qual a diferença fundamental entre cookie e session? Onde cada um guarda os dados? Qual tem limite de tamanho?

O cookie guarda os dados no navegador do usuário (cliente) e tem um limite estrito de até 4 KB, sendo ideal para preferências simples. A session armazena os dados no servidor, não tem limite de tamanho fixo e é muito mais segura, pois envia apenas um código de identificação temporário ao navegador para manter o usuário conectado.

**2.** O que é o cookie connect.sid que aparece no DevTools quando usamos express-session? Ele contém os dados do carrinho/login?

O connect.sid é apenas a chave (Session ID) que identifica o usuário. Ele não contém o carrinho ou o login; esses dados ficam guardados em segurança no servidor. O navegador apenas envia essa chave para o Express saber de quem é o "armário" de dados.

**3.** Explique o fluxo completo de uma session em 5 passos: desde a primeira visita até o servidor ler os dados.

Na primeira visita, o cliente envia uma requisição limpa ao servidor, que cria uma sessão na memória e gera um identificador único. O servidor responde enviando esse ID criptografado para o navegador através do cookie connect.sid. A partir daí, o navegador armazena esse cookie e o reenvia automaticamente em todas as próximas requisições. Por fim, quando o usuário faz uma nova ação, o servidor lê o connect.sid recebido, localiza os dados correspondentes na memória e os disponibiliza no req.session para a aplicação usar.

**4.** Para que serve cada opção da configuração do express-session?

**a)** secret

É a chave secreta usada para criptografar e assinar o cookie connect.sid. Ela garante que o usuário não consiga alterar o ID da sessão para tentar invadir a conta de outra pessoa.

**b)** resave: false

Evita que a sessão seja salva de volta no servidor se ela não foi modificada durante a requisição. Deixar como false poupa desempenho e evita conflitos quando o usuário abre várias abas.

**c)** saveUninitialized: false

Impede que sessões vazias (sem nenhum dado salvo, como logins ou carrinhos) sejam guardadas no servidor. Deixar como false economiza muita memória e evita criar cookies desnecessários para visitantes anônimos.

**d) cookie.maxAge**

Define o tempo de vida do cookie no navegador (em milissegundos). Por exemplo, 60000 dura 1 minuto. Se não for definido, o cookie expira assim que o usuário fecha o navegador.

**e) cookie.httpOnly**

Garante que o cookie só possa ser acessado pelo protocolo HTTP/HTTPS. Definido como true, ele impede que scripts maliciosos de JavaScript (ataques XSS) roubem o cookie no navegador.

**5. O que é declare module e por que precisamos dele ao usar sessions com TypeScript? O que acontece se não criar o arquivo [session.d.ts](#)?**

O declare module serve para injetar novas propriedades em bibliotecas de terceiros. Precisamos dele porque o express-session não sabe quais dados seu app vai guardar (como userId ou carrinho).

Se você não criar o arquivo session.d.ts, o TypeScript vai gerar um erro de compilação e travar o build sempre que você tentar ler ou salvar algo em req.session, acusando que aquela propriedade não existe na tipagem original da biblioteca.

**6. O que é uma flash message? Descreva o padrão de 3 passos: salvar, ler e consumir. Por que o passo "consumir" (setar null) é obrigatório?**

Uma flash message é um alerta temporário, como uma mensagem de sucesso ou erro, exibido apenas uma vez. No passo salvar, o servidor guarda o texto na sessão e redireciona o usuário. No passo ler, a nova página carrega e o servidor resgata esse texto para exibir na tela. No passo consumir, o servidor apaga o dado da sessão imediatamente após a leitura. Esse último passo é obrigatório porque a sessão é persistente; se você não limpar a mensagem, ela ficará presa e reaparecerá na tela do usuário toda vez que ele atualizar a página ou navegar pelo site.

**7. Qual a diferença entre req.session.destroy() e res.clearCookie()? Quando usar cada um?**

req.session.destroy() apaga os dados guardados no servidor, inutilizando a chave do usuário. É usado no logout para garantir que a sessão foi encerrada no backend.

res.clearCookie('connect.sid') ordena que o navegador delete o cookie da chave. Usado isolado, ele desloga o usuário visualmente, mas mantém os dados órfãos ocupando memória no servidor.

A boa prática para um logout perfeito é usar ambos juntos: você destrói os dados no servidor e limpa a chave antiga do navegador.

**8. Por que é seguro guardar dados de login e carrinho na session, mas NÃO é seguro guardá-los em cookies?**

Guardar login e carrinho em cookies é inseguro porque eles ficam armazenados diretamente no computador do usuário, ficando expostos a vírus, extensões maliciosas e ataques de scripts (XSS). Além disso, o usuário poderia facilmente alterar o valor do cookie no navegador para forçar o login na conta de outra pessoa ou modificar os preços do carrinho. Já a session é segura porque mantém todos os dados reais trancados dentro do servidor, em um ambiente controlado pelo desenvolvedor. O navegador recebe apenas o connect.sid, que é uma chave criptografada sem nenhum dado pessoal ou sensível. Mesmo que alguém intercepte essa chave, não conseguirá alterar as informações do sistema, pois toda a validação e lógica de negócio acontecem direto no backend, longe do alcance do cliente.

## Parte 2 — Leitura e análise de código

9. Analise a configuração abaixo. Quanto tempo a sessão dura? O cookie é acessível via document.cookie?

```
app.use(session({
  secret: "minha-chave-2026",
  resave: false,
  saveUninitialized: false,
  cookie: {
    maxAge: 60 * 60 * 1000,
    httpOnly: true,
    secure: false,
  }
}));
```

Quanto tempo a sessão dura? A sessão dura 1 hora. O valor de maxAge está definido em milissegundos:  $60 \times 60 \times 1000 = 3.600.000$  ms, o que equivale exatamente a 60 minutos. Após esse tempo de inatividade, o cookie expira automaticamente no navegador. O cookie é acessível via document.cookie? Não. Como a propriedade httpOnly está configurada como true, o navegador bloqueia qualquer tentativa de leitura ou modificação do cookie via JavaScript no lado do cliente. Isso protege o seu connect.sid contra roubo por scripts maliciosos (ataques XSS).

10. O código abaixo tenta usar req.session.usuario mas dá ERRO no TypeScript. Qual é o erro e como corrigir?

```
// src/routes/minhaRota.ts
app.get("/perfil", (req, res) => {
  const nome = req.session.usuario;
  // ✗ Property "usuario" does not exist on type "SessionData"
  res.send(`Olá, ${nome}`);
});
```

O erro ocorre porque o TypeScript não sabe quais dados você guardará em req.session. Para ele, o campo usuario não existe na tipagem original da biblioteca.

Para corrigir, basta criar um arquivo session.d.ts (geralmente na pasta src/@types/) para estender a interface da biblioteca

*Dica: o que está faltando em src/types/session.d.ts?*

**11.** O código abaixo tem um BUG com flash messages. Qual é? O que o usuário vai ver de errado?

```
app.get("/produtos", (req, res) => {
  const flash = req.session.flash;
  // falta algo aqui...
  res.render("produtos", { flash });
});
```

O bug é que o código lê a mensagem da sessão, mas não a apaga em seguida. Sem o passo de consumo, o dado fica gravado no servidor por tempo indeterminado. Como resultado, o usuário verá o mesmo alerta preso na tela de forma errada; toda vez que ele atualizar a página /produtos ou navegar pelo site e voltar para ela, a mensagem antiga continuará reaparecendo em vez de sumir após a primeira exibição.

**12.** Analise os dois trechos. Qual a diferença entre eles? Qual é o correto para fazer logout?

```
// Trecho A:
app.get("/sair", (req, res) => {
  req.session.usuario = undefined;
  req.session.carrinho = undefined;
  res.redirect("/");
});
```

```
// Trecho B:
app.get("/sair", (req, res) => {
  req.session.destroy(() => {
    res.redirect("/");
  });
});
```

A diferença entre os dois trechos está em apenas limpar as variáveis versus excluir a sessão por completo. No Trecho A, o código apenas esvazia os dados de usuario e carrinho, mas mantém a sessão ativa e o cookie connect.sid totalmente válido no navegador do cliente. Já no Trecho B, o método req.session.destroy() deleta o registro inteiro da sessão direto da memória do servidor, fazendo com que aquela chave do navegador deixe de funcionar instantaneamente no backend. Por esse motivo, o Trecho B é o correto para fazer logout, pois é o único que garante a segurança do sistema ao invalidar o acesso por completo, impedindo que a sessão antiga seja reutilizada caso o cookie seja interceptado.

**13.** O código abaixo adiciona um item ao carrinho. O que acontece se o produto já estiver no carrinho? Qual seria a melhoria?

```
app.post("/carrinho/adicionar/:id", (req, res) => {
  const produto = produtos.find(p => p.id === Number(req.params.id));
  const carrinho = req.session.carrinho || [];
  carrinho.push({ id: produto.id, nome: produto.nome, preco: produto.preco,
    qtd: 1 });
  req.session.carrinho = carrinho;
  res.redirect("/produtos");
});
```

*Dica: se o produto já existir no carrinho, deveria incrementar a quantidade ao invés de duplicar.*

*Se o produto já estiver no carrinho, o código atual adiciona outro item igual, gerando duplicidade. A melhoria é verificar se o produto já existe e, nesse caso, aumentar a propriedade qtd em vez de criar um novo registro. Isso deixa o carrinho mais organizado e facilita o cálculo do total.*

### Parte 3 — Prática: Portal de Notas com Session

Use o projeto portal-notas fornecido pelo professor. As views EJS e o CSS já estão PRONTOS.

Tema: Portal de Notas — o aluno entra com seu nome, adiciona notas, vê o boletim com média e situação.

#### Dados na session:

```
interface SessionData {  
  aluno: string;           // nome do aluno  
  notas: number[];         // array de notas (0-10)  
  flash: string | null;    // mensagem temporária  
}
```

No Portal de Notas, a sessão guarda o nome do aluno, um array com as notas e uma mensagem temporária (`flash`). O aluno informa seu nome, adiciona notas, e o sistema calcula a média a partir das notas armazenadas na sessão. Com base na média, exibe a situação do aluno (por exemplo, aprovado se a média for maior ou igual a 7 e reprovado caso contrário). A mensagem `flash` é usada para mostrar avisos ou confirmações temporárias ao usuário.

15. Complete o arquivo src/types/session.d.ts:

- a) Importar "express-session"
- b) Usar declare module para adicionar os 3 campos: aluno, notas, flash
- c) Exportar (o declare module já expande o tipo global)

No arquivo src/types/session.d.ts, importe "express-session" e use declare module para adicionar os campos aluno, notas e flash à sessão:

```
import "express-session";  
  
declare module "express-session" {  
  interface SessionData {  
    aluno: string;  
    notas: number[];  
    flash: string | null;  
  }  
}
```

Não é necessário exportar nada, pois o declare module já faz a extensão global dos tipos.

16. Complete a configuração de session no arquivo src/app.ts:

- a) Importar express-session

- b) Registrar `app.use(session({ ... }))` com:
- ```
secret: "portal-notas-senac-2026"
resave: false
saveUninitialized: false
cookie: { maxAge: 30 * 60 * 1000, httpOnly: true }
```

Para completar a configuração de sessão no `src/app.ts`, importe o `express-session` e registre o middleware:

```
import express from "express";
import session from "express-session";

const app = express();

app.use(
  session({
    secret: "portal-notas-senac-2026",
    resave: false,
    saveUninitialized: false,
    cookie: {
      maxAge: 30 * 60 * 1000,
      httpOnly: true,
    },
  })
);
```

Se o arquivo já possui outras configurações (`express.json()`, rotas, etc.), mantenha o `app.use(session(...))` antes das rotas que precisam acessar `req.session`.

## 17. Complete as rotas no arquivo `src/routes/notasRoutes.ts`:

### ROTA 1 — GET / (página de login):

- a) Ler flash da session (ou null)
- b) Consumir flash: `req.session.flash = null`
- c) Renderizar "index" com { flash }

### ROTA 2 — POST /entrar:

- d) Ler `req.body.nome`
- e) Se nome vazio: `flash = "Nome é obrigatório!"`, redirecionar para /
- f) Se válido: `req.session.aluno = nome`, `req.session.notas = []`, `flash = "Bem-vindo!"`
- g) Redirecionar para /boletim

### ROTA 3 — GET /boletim:

- h) Se `!req.session.aluno`: `flash = "Faça login!"`, redirecionar para /
- i) Calcular média: `notas.reduce((s,n) => s+n, 0) / notas.length` (ou 0)
- j) Definir situação: `>= 7 "Aprovado"`, `>= 5 "Recuperação"`, `< 5 "Reprovado"`
- k) Ler e consumir flash

l) Renderizar "boletim" com { aluno, notas, media, situacao, flash }

**ROTA 4 — POST /nota:**

m) Ler Number(req.body.nota)

n) Validar: se  $< 0$  ou  $> 10$ , flash = "Nota deve ser entre 0 e 10"

o) Se válida: push no array req.session.notas, flash = "Nota X adicionada!"

p) Redirecionar para /boletim

**ROTA 5 — POST /limpar:**

q) req.session.notas = []

r) flash = "Notas zeradas!"

s) Redirecionar para /boletim

**ROTA 6 — GET /sair:**

t) req.session.destroy(() => res.redirect("/"))

import { Router } from "express";

const router = Router();

// ROTA 1 — GET /

router.get("/", (req, res) => {

const flash = req.session.flash || null;

req.session.flash = null;

res.render("index", { flash });

});

// ROTA 2 — POST /entrar

router.post("/entrar", (req, res) => {

const nome = req.body.nome?.trim();

if (!nome) {

req.session.flash = "Nome é obrigatório!";

return res.redirect("/");

}

req.session.aluno = nome;

req.session.notas = [];

req.session.flash = "Bem-vindo!";

res.redirect("/boletim");

});

```
// ROTA 3 — GET /boletim
router.get("/boletim", (req, res) => {
  if (!req.session.aluno) {
    req.session.flash = "Faça login!";
    return res.redirect("/");
  }

  const aluno = req.session.aluno;
  const notas = req.session.notas || [];

  const media =
    notas.length > 0
      ? notas.reduce((s, n) => s + n, 0) / notas.length
      : 0;

  let situacao = "Reprovado";

  if (media >= 7) {
    situacao = "Aprovado";
  } else if (media >= 5) {
    situacao = "Recuperação";
  }

  const flash = req.session.flash || null;
  req.session.flash = null;

  res.render("boletim", {
    aluno,
    notas,
    media,
    situacao,
    flash,
  });
});

// ROTA 4 — POST /nota
router.post("/nota", (req, res) => {
  const nota = Number(req.body.nota);

  if (nota < 0 || nota > 10 || Number.isNaN(nota)) {
```



```
req.session.flash = "Nota deve ser entre 0 e 10";  
return res.redirect("/boletim");  
}
```

```
req.session.notas.push(nota);  
req.session.flash = `Nota ${nota} adicionada!`;
```

```
res.redirect("/boletim");  
});
```

```
// ROTA 5 — POST /limpar  
router.post("/limpar", (req, res) => {  
  req.session.notas = [];  
  req.session.flash = "Notas zeradas!";
```

```
  res.redirect("/boletim");  
});
```

```
// ROTA 6 — GET /sair  
router.get("/sair", (req, res) => {  
  req.session.destroy(() => {  
    res.redirect("/");  
  });  
});
```

```
export default router;
```

## Fluxo básico

- ☐ Acessar / → tela de login
- ☐ Digitar nome → Entrar → flash **"Bem-vindo!"** aparece
- ☐ Recarregar /boletim → flash desaparece (foi consumida)
- ☐ Adicionar nota 8.5 → flash **"Nota 8.5 adicionada!"**
- ☐ Adicionar nota 6.0 → média =  $(8.5 + 6.0) / 2 = 7.25$
- ☐ Adicionar nota 4.0 → média = 6.17 (aprox.) e situação passa para **"Recuperação"**

## Validação

- ☐ Adicionar nota -1 → flash **"Nota deve ser entre 0 e 10"**
- ☐ Adicionar nota 15 → flash **"Nota deve ser entre 0 e 10"**
- ☐ Acessar /boletim sem login → redirecionamento para /

## Sessão

- ☐ Abrir F12 → Application → Cookies → existe cookie `connect.sid`
- ☐ Abrir F12 → Console → executar `document.cookie`
  - Resultado esperado: vazio ou sem acesso ao `connect.sid` por causa de `httpOnly: true`
- ☐ Verificar que as notas não aparecem no cookie (somente o ID da sessão é armazenado)
- ☐ Clicar **Sair** → sessão destruída → retorna para login
- ☐ Após sair, acessar `/boletim` → redirecionamento para `/`

## Desafios Bônus

**19.** Adicione uma rota POST `/nota/remover/:index` que remove uma nota específica do array pelo índice. Exiba flash "Nota removida!". Valide que o índice existe.

**20.** Adicione uma funcionalidade de "matéria": o aluno escolhe a matéria (Matemática, Português, Programação) e as notas ficam separadas por matéria. Use um objeto na session: `req.session.notas = { matematica: [8,7], portugues: [9], programacao: [10,8,9] }`.

## Checklist Final

### Session:

- ☐ express-session instalado e configurado no `app.ts`
- ☐ secret definido, `resave: false`, `saveUninitialized: false`
- ☐ `cookie.maxAge + httpOnly: true`

### Tipagem:

- ☐ `session.d.ts` com `declare module`
- ☐ Campos: `aluno (string)`, `notas (number[])`, `flash (string|null)`

### Rotas:

- ☐ GET `/` (login) + POST `/entrar` + GET `/boletim`
- ☐ POST `/nota` (validação 0-10) + POST `/limpar` + GET `/sair`
- ☐ Flash: salvar → ler → consumir (null) em TODAS as rotas que usam
- ☐ Proteção: checar `req.session.aluno` antes de acessar `/boletim`
- ☐ Logout: `req.session.destroy()`